Dustin DiSalle
Lab 2 – ICMP Pinger
CS 4220 - Computer Networks
Dr. Xi Tan

Lab 2 – ICMP Pinger Report

Summary

In this lab, I completed a Python-based ICMP Pinger client using raw sockets. The program sends ICMP Echo Request packets to a host and waits for Echo Reply. I filled in the logic required to receive and parse the reply packet, including extracting the IP header, ICMP header, packet identifier, sequence number, timestamp, and TTL value. The timestamp stored in the packet data is used to calculate the round-trip time for each ping.

I also completed the optional tasks by modifying the program to send multiple ping requests, track successful and lost packets, and report the minimum, maximum, and average round-trip times at the end. In addition, I added parsing for common ICMP error responses such as Destination Network Unreachable, Destination Host Unreachable, and TTL Expired in Transit. This made the program behave more like the standard ping utility while still following the lab skeleton structure.

Key Functions and Their Behaviors

- **checksum():** computes the ICMP checksum, required so the packet is valid when transmitted
- **sendOnePing():** builds the Echo Request packet by setting the type to 8, code to 0, adding a timestamp to payload, computing checksum, and sending to the destination with raw socket
- **receiveOnePing():** waits for a response using **select()**. If data is received before the timeout period, it extracts the IP header and ICMP header from the returned packet. It checks whether the response is an Echo Reply and whether the identifier and sequence# match the request. If so, it reads the timestamp from the reply payload and computes the round-trip time. If the reply is an ICMP error message instead of a normal reply, it maps to a readable error description.
- **doOnePing():** one complete ping operation = creating the raw socket, sending 1 request, waiting for 1 reply, and then closing socket.
- **ping():** repeats for a specified number of packets, prints results, stores the successful RTT values, and prints summary of packet loss and minimum, maximum, and average RTT values.

Validation Evidence

I first tested the program using localhost (127.0.0.1) to verify that the ICMP request and reply process worked correctly on the local machine. After that, I tested the program against multiple remote hosts on different continents. The output showed successful replies for reachable hosts and timeout or ICMP error messages when a destination did not respond. These tests confirmed that the program correctly handled normal replies, timeouts, and several ICMP error cases.

Dustin DiSalle
Lab 2 – ICMP Pinger
CS 4220 - Computer Networks
Dr. Xi Tan

Screenshots of the output for the required hosts are included below as evidence of successful testing:

Local Host (127.0.0.1)	PS C:\Users\ddisa\OneDrive\Desktop\SPRING 2026\CS 4220 – Comp Networks\Lab 2_ICMP Pinger_DiSalle_Dustin> python icmp_pinger.py Pinging 127.0.0.1 [127.0.0.1] using Python: Reply from 127.0.0.1: bytes=36 time=0.69ms TTL=128 Reply from 127.0.0.1: bytes=36 time=0.75ms TTL=128 Reply from 127.0.0.1: bytes=36 time=0.40ms TTL=128 Reply from 127.0.0.1: bytes=36 time=0.79ms TTL=128 Ping statistics for 127.0.0.1: Packets: Sent = 4, Received = 4, Lost = 0 (0% loss) Approximate round trip times in milli-seconds: Minimum = 0.40ms Maximum = 0.79ms Average = 0.66ms
Google (N. America)	PS C:\Users\ddisa\OneDrive\Desktop\SPRING 2026\CS 4220 – Comp Networks\Lab 2_ICMP Pinger_DiSalle_Dustin> python icmp_pinger.py Pinging google.com [142.251.35.142] using Python: Reply from 142.251.35.142: bytes=36 time=5.75ms TTL=117 Reply from 142.251.35.142: bytes=36 time=7.20ms TTL=117 Reply from 142.251.35.142: bytes=36 time=5.75ms TTL=117 Reply from 142.251.35.142: bytes=36 time=6.48ms TTL=117 Ping statistics for 142.251.35.142: Packets: Sent = 4, Received = 4, Lost = 0 (0% loss) Approximate round trip times in milli-seconds: Minimum = 5.75ms Maximum = 7.20ms Average = 6.29ms
Cloudfare (1.1.1.1)	PS C:\Users\ddisa\OneDrive\Desktop\SPRING 2026\CS 4220 – Comp Networks\Lab 2_ICMP Pinger_DiSalle_Dustin> python icmp_pinger.py Pinging 1.1.1.1 [1.1.1.1] using Python: Reply from 1.1.1.1: bytes=36 time=6.46ms TTL=59 Reply from 1.1.1.1: bytes=36 time=7.19ms TTL=59 Reply from 1.1.1.1: bytes=36 time=6.73ms TTL=59 Reply from 1.1.1.1: bytes=36 time=7.92ms TTL=59 Ping statistics for 1.1.1.1: Packets: Sent = 4, Received = 4, Lost = 0 (0% loss) Approximate round trip times in milli-seconds: Minimum = 6.46ms Maximum = 7.92ms Average = 7.08ms

Dustin DiSalle
Lab 2 – ICMP Pinger
CS 4220 - Computer Networks
Dr. Xi Tan

Europe	PS C:\Users\ddisa\OneDrive\Desktop\SPRING 2026\CS 4220 - Comp Networks\Lab 2_ICMP Pinger_DiSalle_Dustin> python icmp_pinger.py Pinging bbc.co.uk [151.101.192.81] using Python: Reply from 151.101.192.81: bytes=36 time=7.47ms TTL=60 Reply from 151.101.192.81: bytes=36 time=7.25ms TTL=60 Reply from 151.101.192.81: bytes=36 time=8.64ms TTL=60 Reply from 151.101.192.81: bytes=36 time=6.21ms TTL=60 Ping statistics for 151.101.192.81: Packets: Sent = 4, Received = 4, Lost = 0 (0% loss) Approximate round trip times in milli-seconds: Minimum = 6.21ms Maximum = 8.64ms Average = 7.39ms
Asia	PS C:\Users\ddisa\OneDrive\Desktop\SPRING 2026\CS 4220 - Comp Networks\Lab 2_ICMP Pinger_DiSalle_Dustin> python icmp_pinger.py Pinging www.tsinghua.edu.cn [101.6.15.66] using Python: Reply from 101.6.15.66: bytes=36 time=203.04ms TTL=45 Reply from 101.6.15.66: bytes=36 time=201.42ms TTL=45 Reply from 101.6.15.66: bytes=36 time=209.34ms TTL=45 Reply from 101.6.15.66: bytes=36 time=201.94ms TTL=45 Ping statistics for 101.6.15.66: Packets: Sent = 4, Received = 4, Lost = 0 (0% loss) Approximate round trip times in milli-seconds: Minimum = 201.42ms Maximum = 209.34ms Average = 203.93ms
Africa	PS C:\Users\ddisa\OneDrive\Desktop\SPRING 2026\CS 4220 - Comp Networks\Lab 2_ICMP Pinger_DiSalle_Dustin> python icmp_pinger.py Pinging www.uct.ac.za [137.158.159.192] using Python: Request timed out. Request timed out. Request timed out. Request timed out. Ping statistics for 137.158.159.192: Packets: Sent = 4, Received = 0, Lost = 4 (100% loss)

References

- Kurose, J. F., & Ross, K. W. (2021). *Computer networking: A top-down approach* (8th ed.). Pearson.
- OpenAI. (2026). *ChatGPT (GPT-5.3) [Large language model]*. https://chat.openai.com
  - Used to correct errors in code
  - Helped provide a shortened overall understanding of the lab assignment
- Postel, J. (1981). *Internet Control Message Protocol (ICMP)* (RFC 792). Internet Engineering Task Force. https://doi.org/10.17487/RFC0792
- Python Software Foundation. (2024). *socket — Low-level networking interface*. https://docs.python.org/3/library/socket.html
- Python Software Foundation. (2024). *struct — Interpret bytes as packed binary data*. https://docs.python.org/3/library/struct.html